

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

— o0o —



ĐỒ ÁN LẬP TRÌNH TÍNH TOÁN

**XÂY DỰNG CHƯƠNG TRÌNH TÌM ĐƯỜNG ĐI
NGẮN NHẤT TRÊN ĐỒ THỊ CÓ HƯỚNG CÓ
TRỌNG SỐ**

Người hướng dẫn: ThS. TRẦN HỒ THỦY TIÊN

Sinh viên thực hiện:

Trần Văn Trường Vũ Lớp: 25T_DT2 Nhóm: 25Nh.11A

Trần Ngọc Bảo Quyên Lớp: 25T_DT2 Nhóm: 25Nh.11A

Đà Nẵng, 06/2026

MỤC LỤC

MỤC LỤC	i
DANH MỤC HÌNH VẼ	iii
MỞ ĐẦU	1
1 Tổng quan đề tài	2
1.1 Bài toán tìm đường đi ngắn nhất và tầm quan trọng	2
1.2 Tổng quan về mô phỏng đồ họa thời gian thực	4
1.3 Mục đích thực hiện đề tài	6
1.4 Mục tiêu đề tài	6
1.5 Phạm vi và đối tượng nghiên cứu	6
1.6 Phương pháp nghiên cứu	7
2 Cơ sở lý thuyết	8
2.1 Ý tưởng	8
2.2 Các khái niệm nền tảng	8
2.2.1 Tổng quan về lý thuyết đồ thị (Graph Theory)	8
2.2.2 Lý thuyết đồ thị có hướng (Directed Graph)	8
2.2.3 Nguyên lý nới lỏng cạnh (Relaxation)	9
2.3 Thuật toán Dijkstra	10
2.3.1 Giới thiệu	10
2.3.2 Nguyên lý	10
2.3.3 Các bước thực hiện	10
2.4 Lý thuyết đồ họa và tính toán tọa độ vector	11
3 Tổ chức cấu trúc dữ liệu và thuật toán	12
3.1 Phát biểu bài toán	12
3.2 Cấu trúc dữ liệu	12
3.3 Thuật toán	14
4 Chương trình và kết quả	16
4.1 Tổ chức chương trình	16
4.1.1 Cấu trúc thư mục dự án	16
4.1.2 Chi tiết các hàm xử lý trong mã nguồn	16
4.2 Ngôn ngữ và thư viện cài đặt	17

4.3	Kết quả	18
4.3.1	Giao diện chính của chương trình	18
4.3.2	Kết quả thực thi của chương trình	20
4.3.3	Nhận xét đánh giá	21
5	Kết luận và hướng phát triển	23
5.1	Kết luận	23
5.2	Hướng phát triển	23
	TÀI LIỆU THAM KHẢO	24
	PHỤ LỤC	25

DANH MỤC HÌNH VẼ

4.1	Giao diện chính giới thiệu thông tin đồ án trên màn hình Console . . .	19
4.2	Giao diện đồ họa phân bố hệ thống các đỉnh theo quỹ đạo đồng tâm .	19
4.3	Bảng điều khiển nhật ký hoạt động cập nhật trạng thái loang nhân từng bước	20
4.4	Biểu diễn lộ trình ngắn nhất bằng hiệu ứng đổi màu cung liên kết sang sắc đỏ	21

MỞ ĐẦU

Sự phát triển mạnh mẽ của lý thuyết đồ thị đã đặt nền móng vững chắc cho nhiều bước tiến đột phá trong khoa học máy tính, tối ưu hóa hệ thống và quy hoạch mạng lưới giao thông. Trong số các bài toán kinh điển, bài toán tìm đường đi ngắn nhất trên đồ thị có hướng có trọng số nổi lên như một thách thức trọng tâm với vô vàn ứng dụng thực tiễn—từ việc định tuyến trong mạng Internet, tối ưu hóa lộ trình trong các ứng dụng bản đồ số, đến việc lập lịch trình vận tải và logistics. Việc xác định lộ trình tối ưu không chỉ đơn thuần là tìm kiếm khoảng cách vật lý nhỏ nhất, mà còn liên quan trực tiếp đến việc tối ưu hóa thời gian, chi phí và tài nguyên trong các môi trường mạng lưới có tính chất phân cấp phức tạp.

Đề tài "**Xây dựng chương trình tìm đường đi ngắn nhất trên đồ thị có hướng có trọng số**" theo phương pháp Dijkstra được thực hiện nhằm hiện thực hóa các cơ sở lý thuyết vào thực tiễn lập trình. Mục tiêu trọng tâm của đề tài là nghiên cứu chuyên sâu về nguyên lý hoạt động, các bất biến toán học của thuật toán Dijkstra, từ đó thiết kế và cài đặt một chương trình ứng dụng hoàn chỉnh, có khả năng xử lý chính xác và tối ưu các dữ liệu trên đồ thị có hướng.

Nghiên cứu này tập trung phân tích cấu trúc của đồ thị có hướng có trọng số dương, cấu trúc dữ liệu tối ưu cho thuật toán, và quy trình từng bước cập nhật khoảng cách. Để đảm bảo tính khách quan và hiệu quả vận hành, chương trình được triển khai thực nghiệm trên các bộ dữ liệu mô phỏng khác nhau, kết hợp các kỹ thuật trực quan hóa kết quả để người dùng dễ dàng theo dõi tiến trình của thuật toán. Cấu trúc đề tài được tổ chức tuyến tính từ cơ sở lý thuyết, phân tích thiết kế thuật toán, cài đặt chương trình, đến kiểm chứng thực nghiệm, rút ra kết luận và hướng phát triển.

Do thời gian hạn hẹp và trình độ chuyên môn còn hạn chế, chúng em nhận thức rằng đồ án không tránh khỏi những thiếu sót. Chúng em xin chân thành cảm ơn sự hướng dẫn tận tâm của cô ThS. Trần Hồ Thuỷ Tiên trong suốt quá trình thực hiện đề tài. Nhóm chúng em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để hoàn thiện hơn nữa đề tài này.

Nhóm sinh viên thực hiện

1. Tổng quan đề tài

1.1. Bài toán tìm đường đi ngắn nhất và tầm quan trọng

Trong toán học và lý thuyết đồ thị, Bài toán tìm đường đi ngắn nhất là việc tìm kiếm một đường đi giữa hai đỉnh (nút) trong một đồ thị sao cho tổng trọng số (chi phí, độ dài, thời gian, v.v.) của các cạnh tạo nên đường đi đó là nhỏ nhất.

Định nghĩa toán học một cách hình thức: Cho một đồ thị có trọng số với tập đỉnh V , tập cạnh E , và một hàm trọng số $f : E \rightarrow \mathbb{R}$. Với hai đỉnh bất kỳ v và v' , cần tìm một đường đi P nối từ v đến v' sao cho tổng trọng số $\sum_{p \in P} f(p)$ đạt giá trị nhỏ nhất so với tất cả các đường đi khác có thể nối hai đỉnh này.

1. Các biến thể của bài toán

Bài toán này được chia thành các dạng chính dựa trên nhu cầu tìm kiếm:

- Đường đi ngắn nhất nguồn đơn (Single-source shortest path):** Tìm đường đi ngắn nhất từ một đỉnh xuất phát (nguồn) đến tất cả các đỉnh còn lại trong đồ thị.
- Đường đi ngắn nhất mọi cặp đỉnh (All-pairs shortest path):** Tìm đường đi ngắn nhất giữa mọi cặp đỉnh bất kỳ trong hệ thống.
- Đường đi ngắn nhất đích đơn (Single-destination shortest path):** Tìm đường đi từ tất cả các đỉnh khác hướng về chung một đích đến.
- Đường đi ngắn nhất giữa một cặp đỉnh (Single-pair shortest path):** Tìm đường đi nối từ đỉnh A đến đỉnh B cụ thể.

2. Các thuật toán tiêu biểu

Để giải quyết bài toán này, các nhà khoa học máy tính đã phát triển nhiều thuật toán tối ưu cho các trường hợp khác nhau:

- **Thuật toán Dijkstra:** Là thuật toán nổi tiếng nhất giải quyết bài toán nguồn đơn trên đồ thị có trọng số không âm. Thuật toán này lan truyền từ điểm xuất phát, luôn ưu tiên mở rộng đường đi có chi phí thấp nhất hiện tại.
- **Thuật toán Bellman-Ford:** Cũng giải quyết bài toán nguồn đơn nhưng mạnh mẽ hơn Dijkstra ở chỗ nó có thể xử lý các đồ thị có trọng số âm (ví dụ: các giao dịch tài chính có lợi nhuận/thua lỗ).

- **Giải thuật A* (A-Star):** Là phiên bản nâng cấp của Dijkstra, sử dụng thêm hàm ước lượng (heuristics) để hướng việc tìm kiếm về phía đích đến nhanh hơn. Đây là thuật toán tiêu chuẩn trong phát triển game và bản đồ số.
- **Thuật toán Floyd-Warshall:** Được sử dụng để giải quyết bài toán đường đi ngắn nhất cho mọi cặp đỉnh.
- **Thuật toán Johnson:** Tương tự Floyd-Warshall nhưng hoạt động hiệu quả và nhanh hơn đáng kể trên các "đồ thị thưa"(đồ thị có ít cạnh liên kết).

3. Tầm quan trọng và Ứng dụng thực tiễn

Tuy xuất phát từ toán học lý thuyết, Bài toán tìm đường đi ngắn nhất là "trái tim" của vô số công nghệ cốt lõi trong thời đại số hóa ngày nay.

1. **Định tuyến Mạng và Internet (Telecommunications & Networking):** Trong viễn thông, bài toán này thường được hiểu là "bài toán đường đi có độ trễ nhỏ nhất". Các giao thức định tuyến nền tảng của Internet như OSPF (Open Shortest Path First) hay IS-IS đều sử dụng thuật toán Dijkstra để tìm ra đường truyền gói tin nhanh nhất giữa các bộ định tuyến (router) trên toàn cầu.
2. **Hệ thống dẫn đường & Bản đồ (GPS Navigation):** Google Maps, Apple Maps, hay Grab đều giải quyết bài toán này mỗi khi bạn yêu cầu chỉ đường. Các mạng lưới đường phố chính là đồ thị, các giao lộ là đỉnh, khoảng cách hoặc thời gian di chuyển dự kiến là trọng số.
3. **Trí tuệ nhân tạo và Video Game (Pathfinding):** Khi một nhân vật máy (NPC) trong game di chuyển để đuổi theo bạn trong một bản đồ đầy chướng ngại vật, trò chơi đang chạy thuật toán A* nhiều lần mỗi giây để liên tục tìm đường đi ngắn nhất mà không bị kẹt vào tường.
4. **Logistics và Tối ưu chuỗi cung ứng:** Giúp các công ty giao hàng lập kế hoạch lộ trình cho đội xe sao cho tiết kiệm xăng nhất và giao được nhiều hàng nhất (thường được mở rộng thành Bài toán người bán hàng - Traveling Salesman Problem, một bài toán NP-đầy đủ rất phức tạp).
5. **Mạng xã hội:** Tìm hiểu khoảng cách kết nối giữa hai người trên mạng lưới hàng tỷ người dùng (Ví dụ: Bạn cách Elon Musk bao nhiêu lượt kết nối bạn bè chung).

1.2. Tổng quan về mô phỏng đồ họa thời gian thực

1. Tổng quan về Mô phỏng đồ họa thời gian thực là gì?

Trong lĩnh vực khoa học máy tính, Mô phỏng đồ họa thời gian thực (Real-time graphics simulation) là quá trình máy tính tính toán và xuất ra các hình ảnh liên tục ở tốc độ cao (thường từ 30 đến 60 khung hình/giây trở lên) để phản hồi ngay lập tức với các thay đổi của dữ liệu hoặc tương tác từ người dùng. Khi áp dụng vào thuật toán, thay vì chỉ in ra các dòng chữ, log hoặc con số tĩnh trên màn hình Console (Terminal) truyền thống, mô phỏng đồ họa sẽ "vẽ" lại toàn bộ trạng thái của cấu trúc dữ liệu, bộ nhớ hoặc logic cốt lõi tại thời điểm đó dưới dạng hình học, màu sắc và chuyển động.

2. Tại sao nên dùng đồ họa thay thế Console khô khan?

Việc chuyển đổi từ giao diện dòng lệnh (CLI) sang mô phỏng trực quan mang lại những lợi ích đột phá trong việc hiểu và phân tích thuật toán:

- **Trực quan hóa luồng dữ liệu:** Con người tiếp thu hình ảnh nhanh hơn văn bản. Nhìn các cột cao thấp hoán đổi vị trí cho nhau dễ hiểu và ghi nhớ hơn rất nhiều so với việc đọc hàng trăm dòng mảng số nguyên in ra từ Terminal.
- **Kiểm soát và Tương tác (Interactivity):** Người dùng có thể tua chậm (slow-motion), tạm dừng (pause), hoặc chạy từng bước (step-by-step) quá trình thuật toán thực thi để quan sát sự thay đổi trạng thái—điều rất khó làm mượt mà trên Terminal.
- **Phát hiện lỗi trực giác (Visual Debugging):** Khi thuật toán có lỗi logic (ví dụ: vòng lặp vô hạn, hoặc mảng trỏ sai vùng nhớ), biểu hiện trên đồ họa thường là các chuyển động sai lệch hoặc biến dạng hình ảnh, giúp lập trình viên khoanh vùng lỗi ngay lập tức.
- **Tăng cảm hứng học tập:** Biến những lý thuyết toán học, các vòng lặp for/while khô khan thành các chuyển động bắt mắt, tạo sự hứng thú đặc biệt cho cả người mới học lập trình lẫn kỹ sư lâu năm.

3. Các thuật toán thường được mô phỏng đồ họa

Bất kỳ thuật toán nào cũng có thể hình ảnh hóa, nhưng mang lại hiệu ứng thị giác và giá trị học thuật cao nhất thường là:

- **Thuật toán Sắp xếp (Sorting Algorithms):** Quick Sort, Merge Sort, hay Bubble Sort thường được mô phỏng bằng các cột hoặc thanh ngang có chiều cao

khác nhau. Đồ họa giúp thấy rõ cách mỗi thuật toán "chia để trị" hoặc làm "nổi bật" phần tử lớn nhất.

- **Thuật toán Tìm đường (Pathfinding):** Các thuật toán như Dijkstra, A* hay BFS thường được thể hiện qua dạng lưới (Grid), với các khối màu sắc lan tỏa dần như nước chảy để minh họa không gian tìm kiếm và đường đi ngắn nhất.
- **Duyệt Cây và Đồ thị (Tree & Graph Traversals):** Mô phỏng bằng các nút mạng (node) nối với nhau. Các mảng màu nhấp nháy chuyển từ đỉnh này sang đỉnh khác, vẽ ra nhánh đi của cấu trúc dữ liệu.
- **Hình học máy tính (Computational Geometry):** Thuật toán tìm bao lồi (Convex Hull), phân mảnh không gian (Quadtree) hoặc phát hiện va chạm (Collision Detection) dựa hoàn toàn vào hệ tọa độ 2D/3D.

4. Công cụ và Công nghệ phổ biến

Để xây dựng các ứng dụng mô phỏng này, các nhà phát triển thường sử dụng các thư viện hỗ trợ vẽ đồ họa phần cứng hoặc phần mềm:

- **Nền tảng Web (Chạy trên trình duyệt):** HTML5 Canvas, p5.js, WebGL, Three.js (dành cho 3D). Đây là cách phổ biến nhất hiện nay vì tính chia sẻ cao, người dùng không cần cài đặt phần mềm.
- **Môi trường Python:** Pygame, Matplotlib, hoặc Manim (thư viện chuyên dụng tạo animation toán học cực kỳ nổi tiếng).
- **C/C++ & Hiệu năng cao:** OpenGL, Raylib (đây là thư viện nhóm chúng em đã sử dụng để phát triển đồ án này), SFML, hoặc ứng dụng thẳng vào các Game Engine như Unity, Unreal Engine, Godot để tận dụng sức mạnh GPU.

5. Tầm quan trọng và Ứng dụng thực tiễn

1. **Công nghệ Giáo dục (EdTech):** Các nền tảng học thuật như LeetCode, HackerRank hoặc các trường đại học đang ngày càng tích hợp nhiều công cụ mô phỏng để sinh viên dễ dàng nắm bắt các khái niệm trừu tượng như con trỏ (pointers), đệ quy (recursion).
2. **Hỗ trợ Nghiên cứu Khoa học:** Các mô phỏng về dịch tế học (sự lây lan mầm bệnh), mô phỏng hạt (Particle systems), hay Cellular Automata (như Game of Life) đều cần đồ họa để các nhà nghiên cứu "nhìn thấy" kết quả hình mẫu từ các phương trình phức tạp.

3. **Phát triển Trò chơi (Game Development):** Trực quan hóa các thuật toán vật lý (mô phỏng trọng lực, chất lỏng, ánh sáng) và AI để tinh chỉnh logic trước khi đưa vào sản phẩm thực tế.
4. **Thuyết trình và Trình diễn Kỹ thuật:** Một video hoặc ứng dụng tương tác trực tiếp luôn có sức thuyết phục và giúp truyền đạt ý tưởng nhanh hơn hàng chục trang tài liệu mô tả mã nguồn thuần túy.

1.3. Mục đích thực hiện đề tài

Trong kỷ nguyên công nghệ số và sự phát triển mạnh mẽ của các hệ thống mạng lưới, bài toán tối ưu hóa đường đi trên đồ thị đóng vai trò cốt lõi trong nhiều lĩnh vực thực tiễn. Phần mềm hỗ trợ trực quan hóa giúp công việc thiết kế, phân tích và học tập trở nên tối ưu và thân thiện hơn. Đề tài này được thực hiện nhằm mục đích giúp nhóm sinh viên rèn luyện tư duy thiết kế giải thuật hợp lý, hiện thực hóa các kiến thức cốt lõi đã học trong học phần Toán rời rạc, Kỹ thuật lập trình và Cấu trúc dữ liệu để giải quyết một bài toán thực tiễn phức tạp.

1.4. Mục tiêu đề tài

Mục tiêu trung tâm của đồ án là phân tích chuyên sâu, thiết kế và thực thi ứng dụng tìm đường đi ngắn nhất giữa hai điểm bất kỳ trên đồ thị có hướng có trọng số. Các mục tiêu cụ thể bao gồm:

- **Xử lý dữ liệu linh hoạt:** Thiết lập module tự động đọc và phân tích cấu trúc ma trận kề từ tệp tin văn bản định dạng tĩnh.
- **Cài đặt thuật toán chính xác:** Áp dụng và tối ưu hóa giải thuật Dijkstra để tính toán nhãn đường đi tối ưu, đồng thời lưu vết phục vụ truy xuất lộ trình logic.
- **Mô phỏng đồ họa trực quan:** Tích hợp thư viện đồ họa Raylib để thiết kế không gian tương tác động, biểu diễn quá trình loang duyệt đỉnh một cách sinh động nhất.

1.5. Phạm vi và đối tượng nghiên cứu

- **Đối tượng nghiên cứu:** Cơ sở toán học của đồ thị có hướng, thuật toán tối ưu hóa nhãn Dijkstra, cấu trúc dữ liệu ma trận kề, phương trình hình học tọa độ vector và nguyên lý máy trạng thái (State Machine).

- **Phạm vi nghiên cứu:** Vận dụng tổng hợp ngôn ngữ lập trình C/C++ thuần túy kết hợp thư viện Raylib. Giới hạn trên mô hình đồ thị có trọng số cạnh không âm (điều kiện hội tụ của thuật toán Dijkstra).

1.6. Phương pháp nghiên cứu

Đồ án vận dụng phương pháp nghiên cứu lý thuyết kết hợp thực nghiệm. Nhóm đã tiến hành khảo sát các giáo trình Toán rời rạc, nghiên cứu tài liệu lập trình đồ họa tương tác. Từ đó, xây dựng thuật toán, tiến hành code thực nghiệm, đánh giá kết quả chạy trên các tập dữ liệu giả lập và tinh chỉnh logic hiển thị để đạt được trải nghiệm quan sát tốt nhất.

2. Cơ sở lý thuyết

2.1. Ý tưởng

Ý tưởng cốt lõi của ứng dụng là xây dựng một hệ thống tách biệt luồng xử lý dữ liệu và luồng hiển thị. Chương trình sẽ đọc ma trận đồ thị từ file, sau đó khởi tạo một hệ thống Máy trạng thái (State Machine) để làm chậm quá trình thực thi của thuật toán Dijkstra. Kết hợp với việc quy hoạch tọa độ hình học, mỗi bước tính toán của thuật toán sẽ được đồng bộ hóa với một khung hình đồ họa (frame), tạo ra hiệu ứng chuyển động và đổi màu trực quan.

2.2. Các khái niệm nền tảng

2.2.1. Tổng quan về lý thuyết đồ thị (Graph Theory)

Trước khi tìm hiểu về thuật toán, điều đầu tiên là cần nắm vững các kiến thức cơ sở về lý thuyết đồ thị.

Định nghĩa và phân loại

Một đồ thị là một cặp các tập (V, E) , trong đó V là tập các đỉnh và E là tập các cạnh, được biểu diễn bởi cặp đỉnh. Tính chất của các mối liên kết (cạnh) giữa các đối tượng (đỉnh) trong bài toán mà đồ thị được phân loại thành:

- **Có hướng hoặc vô hướng:** Xác định tính chất một chiều (digraph) hoặc hai chiều của các liên kết giữa các cặp nút.
- **Có trọng số hoặc không có trọng số:** Mỗi cạnh (u, v) có thể mang trọng số $w(u, v)$ đại diện cho các thực thể vật lý như chi phí di chuyển, khoảng cách vật lý hoặc độ trễ thời gian.

2.2.2. Lý thuyết đồ thị có hướng (Directed Graph)

Đồ thị có hướng được biểu diễn toán học dưới dạng một hệ vô hướng $G = (V, E)$, trong đó:

- $V = \{v_1, v_2, \dots, v_n\}$ là tập hợp hữu hạn các đỉnh của đồ thị.
- $E \subseteq V \times V$ là tập hợp các cung (cạnh có hướng). Mỗi cung là một cặp có thứ tự (u, v) thể hiện liên kết một chiều từ đỉnh gốc u đến đỉnh ngọn v .

- Hàm trọng số $W : E \rightarrow \mathbb{R}^+$ gán cho mỗi cung (u, v) một giá trị $W(u, v)$ đại diện cho khoảng cách hoặc chi phí.

Đồ thị được mã hóa thành "Ma trận kề" $A_{n \times n}$ trong tập tin văn bản đầu vào. Nếu có cung nối từ i đến j , phần tử $A[i][j] = W(i, j)$, ngược lại (không có cung) $A[i][j] = 0$. Khi nạp vào chương trình, giá trị 0 sẽ được chuyển thành -1 để biểu diễn trạng thái không có cạnh nối.

2.2.3. Nguyên lý nới lỏng cạnh (Relaxation)

Kỹ thuật nới lỏng cạnh (Edge Relaxation) là một thao tác cơ sở và cốt lõi trong việc thiết kế và triển khai các thuật toán tìm đường đi ngắn nhất trên đồ thị có hướng có trọng số. Về mặt bản chất, quá trình này liên tục đánh giá và cập nhật lại chi phí (khoảng cách ước lượng) từ đỉnh nguồn đến các đỉnh lân cận cho đến khi hệ thống đạt được trạng thái tối ưu toàn cục.

Ý nghĩa hoạt động

Giả sử trên một đồ thị tồn tại một cạnh nối từ đỉnh u đến đỉnh v . Thao tác nới lỏng kiểm tra xem việc di chuyển từ đỉnh nguồn đến v thông qua đỉnh trung gian u có mang lại tổng chi phí thấp hơn so với đường đi hiện tại đã được ghi nhận hay không. Nếu điều kiện tối ưu này thỏa mãn, hệ thống sẽ loại bỏ chi phí cũ và cập nhật lại giá trị mới tối ưu hơn.

Cơ sở toán học và Cấu trúc logic

Xét đồ thị có trọng số $G = (V, E)$, với V là tập đỉnh và E là tập cạnh.

- Gọi $d(u)$ và $d(v)$ lần lượt là khoảng cách ngắn nhất ước lượng hiện tại từ đỉnh nguồn đến đỉnh u và đỉnh v .
- Gọi $w(u, v)$ là trọng số (chi phí) của cạnh có hướng $(u, v) \in E$.

Thao tác nới lỏng cạnh (u, v) được thực thi thông qua hệ thức điều kiện sau:

Nếu $d(u) + w(u, v) < d(v)$, tiến hành cập nhật:

$$d(v) = d(u) + w(u, v)$$

Đồng thời với việc cập nhật khoảng cách, hệ thống sẽ lưu vết lại cấu trúc đường đi bằng cách gán đỉnh liền trước (predecessor) của v là u . Việc lưu trữ này phục vụ trực tiếp cho quá trình truy xuất lại lộ trình chi tiết từ nguồn đến đích sau khi thuật toán kết thúc quá trình duyệt.

Ứng dụng trong các thuật toán nền tảng

Kỹ thuật nổi lỏng là tư tưởng chủ đạo để xây dựng nên các giải thuật đồ thị quan trọng:

- **Thuật toán Dijkstra:** Giải quyết bài toán đường đi ngắn nhất nguồn đơn trên đồ thị có trọng số không âm ($w(u, v) \geq 0, \forall (u, v) \in E$). Thuật toán áp dụng chiến lược tham lam (Greedy), thực hiện nổi lỏng một cách có chọn lọc từ các đỉnh đã được xác nhận khoảng cách tối ưu.
- **Thuật toán Bellman-Ford:** Xử lý được các đồ thị có chứa cạnh trọng số âm. Thuật toán này áp dụng phương pháp quy hoạch động, tiến hành nổi lỏng toàn bộ tập cạnh E lặp lại $|V| - 1$ lần để đảm bảo mọi đường đi tối ưu đều được bao phủ, đồng thời hỗ trợ phát hiện chu trình âm (negative cycle) trong đồ thị.

2.3. Thuật toán Dijkstra

2.3.1. Giới thiệu

Thuật toán Dijkstra, được đề xuất bởi nhà khoa học máy tính Edsger W. Dijkstra vào năm 1956, là một trong những giải thuật kinh điển và nền tảng nhất để giải quyết bài toán đường đi ngắn nhất nguồn đơn (single-source shortest path) trên đồ thị. Thuật toán này có thể áp dụng trên cả đồ thị có hướng và vô hướng, với điều kiện tiên quyết bắt buộc là trọng số của tất cả các cạnh trong hệ thống phải là các giá trị không âm ($w(u, v) \geq 0$). Trong khuôn khổ của đề án này, Dijkstra được chọn làm nhân xử lý cốt lõi.

2.3.2. Nguyên lý

Thuật toán Dijkstra hoạt động dựa trên chiến lược Tham lam (Greedy Algorithm) kết hợp chặt chẽ với nguyên lý Nổi lỏng cạnh (Edge Relaxation). Tại mỗi bước tiến hành, thuật toán luôn tin tưởng và chọn ra đỉnh có khoảng cách ước lượng từ nguồn đến nó là nhỏ nhất trong tập hợp các đỉnh chưa được xét. Khi một đỉnh được lựa chọn, thuật toán coi khoảng cách đến đỉnh đó đã đạt trạng thái tối ưu tuyệt đối, sau đó sử dụng đỉnh này làm tâm loang để thực hiện "nổi lỏng" (đánh giá và cập nhật lại khoảng cách ngắn hơn) cho tất cả các đỉnh lân cận kề với nó.

2.3.3. Các bước thực hiện

Quy trình thực thi của thuật toán được chia thành các bước cơ bản sau:

1. **Khởi tạo:** Gán nhãn khoảng cách của đỉnh xuất phát $L[u_start] = 0$ và khởi tạo nhãn của tất cả các đỉnh còn lại là vô cực ($L[v] = \infty$). Đánh dấu toàn bộ các đỉnh trong đồ thị ở trạng thái chưa được cố định ($T[v] = 1$).
2. **Tìm đỉnh có nhãn nhỏ nhất:** Quét qua các đỉnh đang có trạng thái $T[i] == 1$ để tìm ra đỉnh u có nhãn khoảng cách $L[u]$ đạt giá trị nhỏ nhất. Nếu không còn đỉnh nào để xét, hoặc đỉnh nhỏ nhất được tìm thấy chính là đỉnh đích cần đến, thuật toán kết thúc quá trình loang.
3. **Nới lỏng cạnh:** Đánh dấu đỉnh u là đã xét xong ($T[u] = 0$). Xét mọi đỉnh v kề trực tiếp với u , nếu phát hiện đường đi qua u ngắn hơn đường đi hiện tại ($L[u] + W(u, v) < L[v]$), tiến hành cập nhật lại $L[v]$ và ghi nhận đỉnh liền trước $par[v] = u$ để phục vụ truy vết.
4. **Lặp lại:** Quay trở lại Bước 2 để tiếp tục chọn đỉnh nhỏ nhất tiếp theo cho đến khi toàn bộ đồ thị được tối ưu hoặc tìm được đích đến.

2.4. Lý thuyết đồ họa và tính toán tọa độ vector

Để hiện thực hóa ứng dụng mô phỏng, hệ thống yêu cầu sự can thiệp của hình học tính toán. Để các đỉnh không đè lên nhau, đồ án áp dụng phương trình tham số đường tròn, dàn đều n đỉnh quanh một tâm $I(x_0, y_0)$ với bán kính R :

$$x_i = x_0 + R \cdot \cos \left((i-1) \frac{2\pi}{n} \right) \quad (2.1)$$

$$y_i = y_0 + R \cdot \sin \left((i-1) \frac{2\pi}{n} \right) \quad (2.2)$$

3. Tổ chức cấu trúc dữ liệu và thuật toán

3.1. Phát biểu bài toán

- **Dữ liệu đầu vào:** Hệ thống nhận tệp cấu trúc văn bản lưu trữ ma trận kề A kích thước $n \times n$. Kèm theo là chỉ số của hai nút bắt đầu và kết thúc lộ trình truy vấn.
- **Yêu cầu đầu ra:** Cửa sổ đồ họa 60-FPS hiển thị quá trình đổi màu động của các đỉnh (Vàng: Đang xét, Xanh: Đã tối ưu) và bảng điều khiển Log ghi nhận từng dòng trạng thái toán học.

3.2. Cấu trúc dữ liệu

Cấu trúc dữ liệu

Hệ thống đóng gói toàn bộ trạng thái lý thuyết vào một `struct Graph` tĩnh, sử dụng hệ thống mảng một chiều nhằm tối ưu hóa bộ nhớ cache để xử lý tốc độ cao. Đồng thời, thuật toán được thiết kế lại dưới dạng Máy trạng thái (State Machine) thông qua enum `AlgoState` để tương thích với giao diện đồ họa.

Dưới đây là mã nguồn cài đặt cấu trúc dữ liệu cốt lõi của chương trình:

```
1 #define N 105
2 #define INF 1000000000
3
4 typedef enum {
5     STATE_FIND_MIN,
6     STATE_RELAX_EDGES,
7     STATE_FINISHED
8 } AlgoState;
9
10 struct Graph {
11     int n; // Số lượng đỉnh
12     int A[N][N]; // Ma trận kề trong sơ
13     Vector2 toado[N]; // Tọa độ trực quan của đỉnh trên UI
14     int L[N]; // Mảng khoảng cách ngắn nhất (Nhân tam thời)
15     int T[N]; // Trạng thái tập hợp T (1: Chưa có đỉnh, 0:
        Da có đỉnh)
16     int par[N]; // Mảng lưu vết cha truy xuất lộ trình
17 };
18
19 struct Node {
```



```

20     int id;                // ID của đỉnh
21     int dist;              // Khoảng cách tại thời điểm thêm vào heap
22 };
23
24 struct priority_queue {
25     Node A[1000];          // Mảng chứa các phần tử của heap
26     int heap_size;         // Kích thước hiện tại của heap
27 };

```

Định nghĩa cấu trúc đồ thị, hàng đợi ưu tiên và máy trạng thái

Chi tiết các thành phần:

- **Hằng số giới hạn:** Hệ thống sử dụng $N = 105$ để giới hạn kích thước mảng tĩnh, ngăn chặn lỗi tràn bộ nhớ. Giá trị $INF = 1000000000$ đại diện cho vô cực, dùng để khởi tạo nhãn khoảng cách ban đầu.
- **AlgoState:** Biến liệt kê quản lý 3 trạng thái của đồ họa:
 - **STATE_FIND_MIN:** Quét tìm đỉnh trong đồ thị có nhãn khoảng cách nhỏ nhất để chọn làm tâm loang.
 - **STATE_RELAX_EDGES:** Chuyển sang trạng thái nối lỏng cạnh, kiểm tra và tối ưu hóa khoảng cách cho các đỉnh kề.
 - **STATE_FINISHED:** Kích hoạt khi tìm thấy đích hoặc đã duyệt xong toàn bộ đồ thị.
- **struct Graph:**
 - **n** và **A[N][N]:** Lưu trữ số lượng đỉnh thực tế và ma trận trọng số của đồ thị.
 - **toado[N]:** Kiểu dữ liệu **Vector2** (kế thừa từ thư viện Raylib) lưu trữ tọa độ (x, y) thực tế trên màn hình để render các đỉnh.
 - **L[N]** và **T[N]:** Hai mảng cốt lõi của thuật toán. L lưu chi phí đường đi, T đánh dấu trạng thái đỉnh (1: chưa cố định, 0: đã tối ưu hoàn toàn).
 - **par[N]:** Mảng lưu vết đỉnh cha liền trước, hỗ trợ truy xuất ngược từ đỉnh đích về đỉnh nguồn sau khi thuật toán kết thúc.
- **struct Node và priority_queue:** Cấu trúc Hàng đợi ưu tiên (Min-Heap) lưu trữ đỉnh kèm theo khoảng cách tại thời điểm được chèn. Việc đóng gói này giúp duy trì cấu trúc Heap ổn định không bị phá vỡ khi khoảng cách toàn cục thay đổi.

3.3. Thuật toán

Dựa trên thuật toán Dijkstra nguyên thủy, nhóm đã tái cấu trúc quy trình loang nhân thành một Máy trạng thái (State Machine) thông qua `enum AlgoState`. Việc xé nhỏ thuật toán thành từng "trạng thái rời rạc" giúp chương trình có thể tạm dừng hoặc làm chậm tiến trình (delay) để đồng bộ với khung hình đồ họa 60-FPS của Raylib.

Quy trình thực thi thực tế trong file `src/main.cpp` diễn ra như sau:

- **Khởi tạo (ResetAlgo):** Toàn bộ đỉnh được gán khoảng cách vô cực `g.L[i] = INF`, riêng đỉnh nguồn `g.L[u_start] = 0`. Trạng thái được đặt về `STATE_FIND_MIN`.
- **Trạng thái tìm đỉnh Min (STATE_FIND_MIN):**
 - Hệ thống sẽ liên tục lấy (pop) đỉnh `top` có khoảng cách nhỏ nhất từ Hàng đợi ưu tiên (Min-Heap). Nếu đỉnh này đã được thăm trước đó (đỉnh rác), hệ thống sẽ bỏ qua.
 - Ngược lại, nếu là đỉnh hợp lệ, hệ thống sẽ chốt đỉnh đó (`g.T[currentV] = 0`) và chuyển sang trạng thái nối lỏng `STATE_RELAX_EDGES`.
 - Nếu tìm thấy đỉnh đích (`currentV == v_target`) hoặc Hàng đợi ưu tiên rỗng, trạng thái chuyển sang `STATE_FINISHED`.
- **Trạng thái nối lỏng (STATE_RELAX_EDGES):**
 - Chương trình không dùng vòng lặp `for` chạy một mạch, mà duyệt từng đỉnh kề `neighborIdx` qua mỗi khung hình (frame).
 - Nếu phát hiện đường đi qua `currentV` ngắn hơn, hệ thống cập nhật lại khoảng cách mới `g.L[neighborIdx]`, ghi nhận đỉnh cha `g.par`, đồng thời **chèn (insert) một bản sao mới** của đỉnh này vào Min-Heap.
 - Khi duyệt xong toàn bộ đỉnh kề của `currentV`, máy trạng thái tự động trả về `STATE_FIND_MIN` để tiếp tục vòng lặp tìm đỉnh tiếp theo.
- **Trạng thái kết thúc (STATE_FINISHED):** Hệ thống dừng quét, tiến hành truy xuất ngược mảng `g.par` từ đỉnh đích về nguồn để nối lại lộ trình, đánh dấu mảng `onPath` thành `true` để hiển thị màu đỏ trên giao diện đồ họa.

Đánh giá độ phức tạp thuật toán

Dựa trên cách cài đặt bằng ma trận kề kết hợp với Hàng đợi ưu tiên (Min-Heap) theo kỹ thuật Lazy Deletion, độ phức tạp của chương trình được phân tích cụ thể như sau:

1. Độ phức tạp thời gian:

- **Thao tác với Min-Heap:** Trong trường hợp xấu nhất, mỗi cạnh đều tạo ra một lần chèn vào Heap. Việc lấy đỉnh và chèn đỉnh tiêu tốn $O(E \log V)$ thời gian.
- **Thao tác nối lỏng (STATE_RELAX_EDGES):** Chương trình cần duyệt qua một hàng của ma trận kề $A[N][N]$ để tìm đỉnh kề. Đối với toàn bộ quá trình, quét ma trận kề tiêu tốn $O(V^2)$ thời gian.

⇒ **Kết luận:** Độ phức tạp thời gian tổng thể của hệ thống là $O(V^2 + E \log V)$. Dù không hoàn toàn đạt $O(E \log V)$ do vẫn bị ràng buộc bởi ma trận kề, việc tích hợp Min-Heap giúp tăng tốc đáng kể các bước chọn đỉnh Min so với thuật toán cơ bản.

2. Độ phức tạp không gian (Bộ nhớ):

- **Lưu trữ đồ thị:** Cấu trúc ma trận kề $A[N][N]$ chiếm phần lớn không gian bộ nhớ, tương đương $O(V^2)$.
- **Lưu trữ Heap và trạng thái:** Hàng đợi ưu tiên có thể chứa tối đa E đỉnh tương đương $O(E)$. Các mảng một chiều $L[N]$, $T[N]$, $par[N]$ chiếm $O(V)$.

⇒ **Kết luận:** Độ phức tạp bộ nhớ của thuật toán là $O(V^2)$, hoàn toàn phù hợp và an toàn trong giới hạn bộ nhớ của môi trường C/C++.

4. Chương trình và kết quả

4.1. Tổ chức chương trình

4.1.1. Cấu trúc thư mục dự án

Để đảm bảo tính khoa học và dễ bảo trì, dự án được tổ chức thành các thư mục riêng biệt với từng vai trò cụ thể như sau:

- **src/**: Chứa mã nguồn C/C++ cốt lõi của chương trình (file `main.cpp`).
- **DATA/**: Thư mục lưu trữ các tệp văn bản đầu vào (ví dụ: `input.dat`) chứa kích thước đỉnh, ma trận kề của đồ thị và đỉnh bắt đầu/kết thúc.
- **lib/ và obj/**: Thư mục hệ thống chứa các thư viện đồ họa Raylib và các file đối tượng sinh ra trong quá trình biên dịch.
- **Makefile**: Tập lệnh được cấu hình sẵn để tự động hóa quá trình biên dịch toàn bộ dự án từ mã nguồn thành file thực thi (`main.exe`).

4.1.2. Chi tiết các hàm xử lý trong mã nguồn

Chương trình được phân rã chức năng mạch lạc thành các module tuyến tính trong tệp tin duy nhất `src/main.cpp`. Cấu trúc này giúp dễ dàng kiểm soát vòng đời của chương trình từ lúc đọc file đến lúc vẽ hình. Dưới đây là chi tiết các hàm đã được cài đặt:

- **main()**: Điểm khởi đầu của chương trình, chứa vòng lặp vô tận (game loop cơ bản) để quản lý luồng thực thi: in giao diện Console, đọc đường dẫn file, kiểm tra tính hợp lệ của file và kích hoạt hàm mô phỏng Raylib đồ họa.
- **UIConsole() & PrintCentered()**: Đảm nhận chức năng in giao diện trang bìa của đồ án trên màn hình dòng lệnh (Terminal), sử dụng các mã màu ANSI để định dạng thẩm mỹ.
- **filename() & inputdata()**: Cụm hàm giao tiếp với người dùng để lấy đường dẫn tệp tin đầu vào. Hàm `inputdata()` chịu trách nhiệm mở file, phân tích kích thước đỉnh, khởi tạo cấu trúc đồ thị **Graph** và chuẩn hóa ma trận kề (chuyển các số 0 thành -1 để biểu thị không có đường đi).

- `UIRaylib_Dijkstra()`: Tráit tìm của ứng dụng đồ họa. Hàm này khởi tạo cửa sổ Raylib 60-FPS, trực tiếp chạy Máy trạng thái (State Machine) thuật toán Dijkstra, tính toán tọa độ phân bố đều các đỉnh trên hình elip/tròn và vẽ toàn bộ trạng thái lên màn hình.
- `drawArrow()`: Hàm hình học nâng cao xử lý toán học lượng giác (`sin`, `cos`, `atan2`) để vẽ chính xác mũi tên chỉ hướng giữa hai tọa độ vector, đồng thời đánh kèm trọng số của cạnh hiển thị ngay chính giữa mũi tên.
- `AddLog()`: Tiện ích hỗ trợ sao chép chuỗi kí tự thông báo tiến trình toán học đẩy vào mảng chuỗi `stepLogs` để vẽ lên màn hình đồ họa bên phải.
- `MyDelay()`: Hàm dừng chương trình cưỡng bức dựa trên xung nhịp đồng hồ máy tính (`clock()`), đóng vai trò làm công cụ tạo nhịp độ chậm cho thuật toán, giúp mắt người dùng kịp quan sát.

4.2. Ngôn ngữ và thư viện cài đặt

Hệ thống được lập trình bằng ngôn ngữ C/C++ trên nền tảng trình biên dịch GCC (MinGW). Công cụ xây dựng chương trình được cấu hình tự động thông qua `Makefile`. Dưới đây là các thư viện được sử dụng để xây dựng ứng dụng:

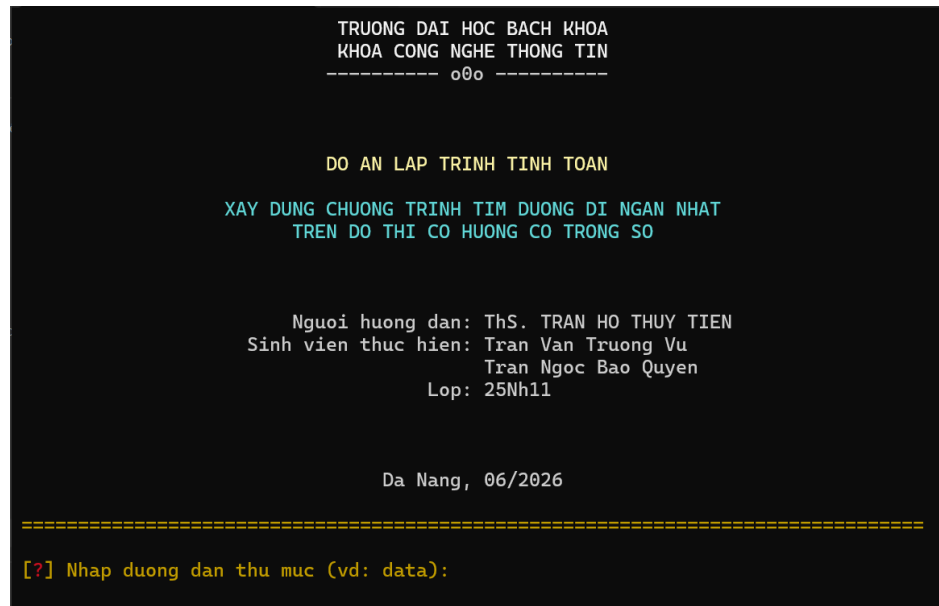
Bảng 4.1: Các thư viện sử dụng trong chương trình

STT	Tên thư viện	Vai trò và chức năng trong chương trình
1	<stdio.h>	Hỗ trợ luồng nhập xuất chuẩn, phục vụ đọc ma trận kề và các tham số cấu hình từ tệp tin văn bản (.dat).
2	<stdlib.h>	Cung cấp các hàm tiện ích hệ thống, trong chương trình này được sử dụng chủ yếu để gọi lệnh xóa màn hình Console (<code>system("cls")</code>).
3	<string.h>	Xử lý và định dạng các chuỗi văn bản, sao chép nhật ký toán học phục vụ bảng hiển thị Log thời gian thực.
4	<math.h>	Cung cấp các hàm lượng giác (<code>sinf</code> , <code>cosf</code> , <code>atan2f</code>) để tính tọa độ đỉnh vòng tròn và xác định góc quay của mũi tên hướng cạnh.
5	<time.h>	Quản lý bộ đếm nhịp xung vật lý, làm cơ sở xây dựng hàm tạo độ trễ (<code>delay</code>) làm chậm tiến trình loang của thuật toán.
6	<raylib.h>	Thư viện đồ họa cốt lõi; quản lý cửa sổ tương tác, duy trì vòng lặp 60-FPS và thực hiện vẽ các thực thể hình học.

4.3. Kết quả

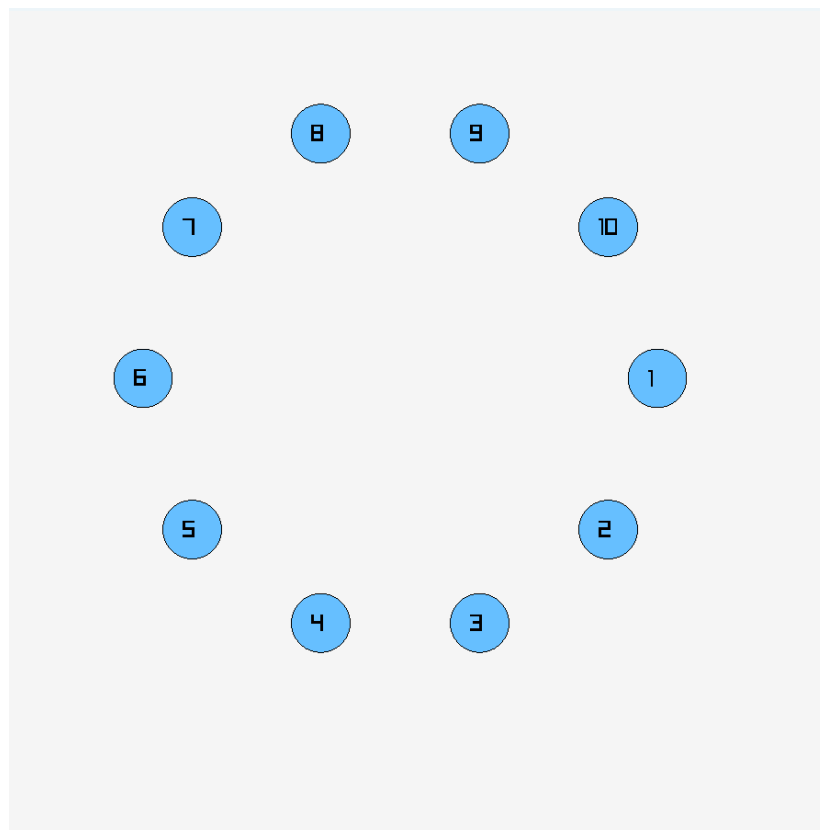
4.3.1. Giao diện chính của chương trình

Màn hình khởi tạo (Console) đảm nhận vai trò hiển thị thông tin sinh viên thực hiện, giới thiệu hệ thống và xác nhận việc nạp tệp đồ thị thành công.



Hình 4.1: Giao diện chính giới thiệu thông tin đồ án trên màn hình Console

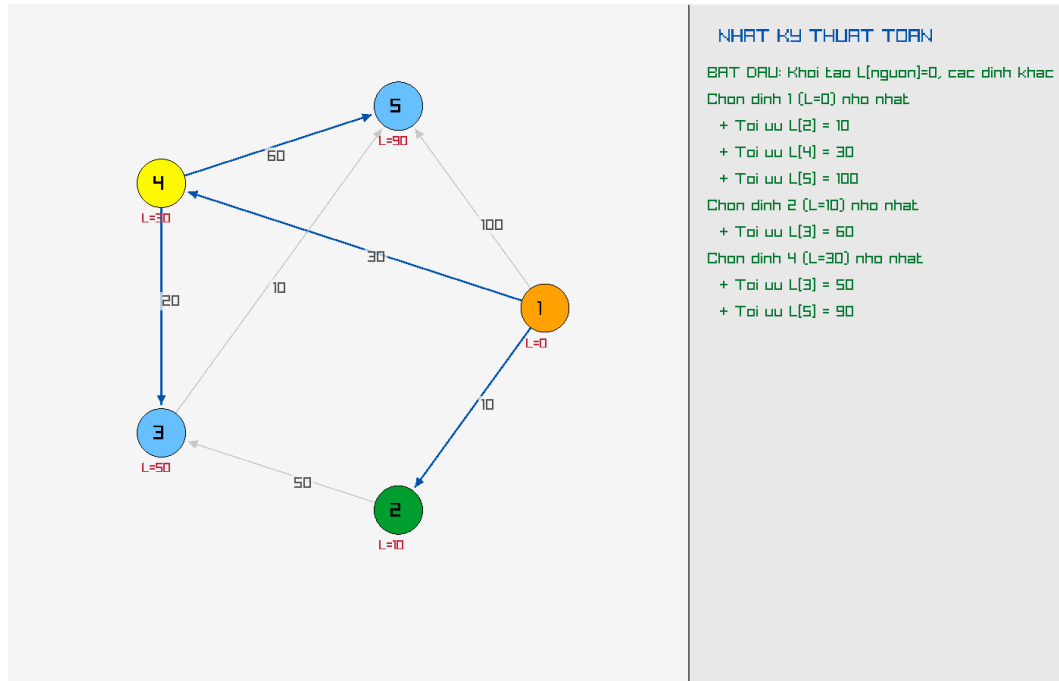
Sau khi đọc dữ liệu thành công, chương trình sẽ khởi chạy với sân khấu phân bố các nút đỉnh đều theo cấu trúc elip tự động.



Hình 4.2: Giao diện đồ họa phân bố hệ thống các đỉnh theo quỹ đạo đồng tâm

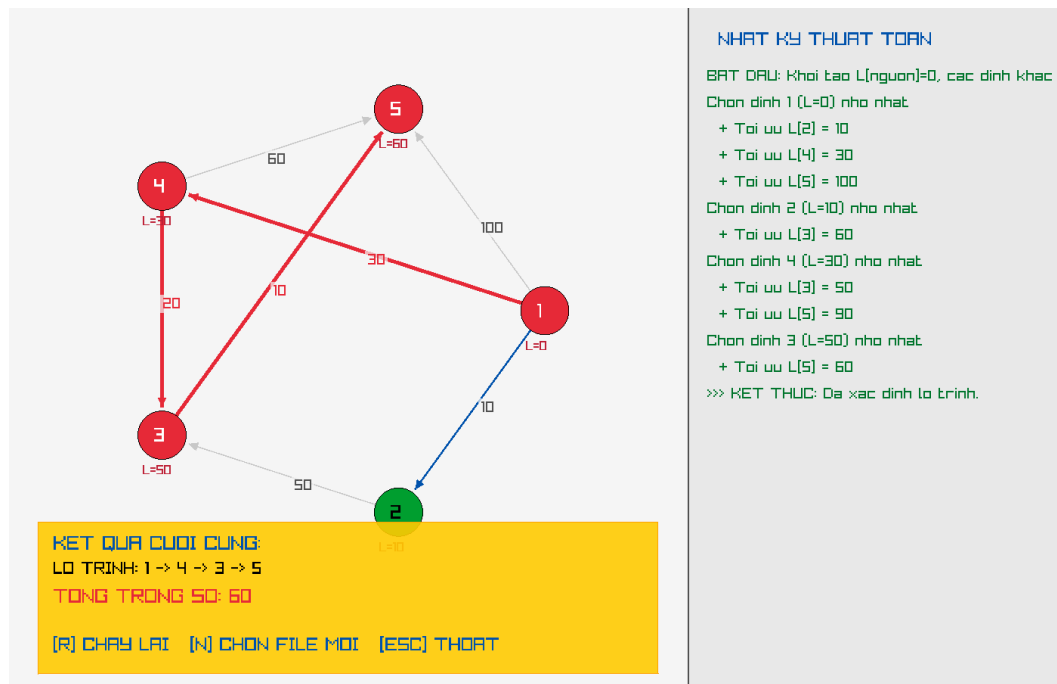
4.3.2. Kết quả thực thi của chương trình

Trong suốt quá trình thuật toán được thực thi, nhãn thông tin khoảng cách L cập nhật liên tục dưới chân từng nút tròn. Sắc xanh lá đại diện cho đỉnh đã tối ưu hoàn toàn, sắc vàng biểu thị đỉnh đang đóng vai trò tâm loang xét duyệt hiện tại. Bất kỳ sự thay đổi nào cũng được ghi nhận vào bảng Log bên phải.



Hình 4.3: Bảng điều khiển nhật ký hoạt động cập nhật trạng thái loang nhãn từng bước

Khi tìm thấy đỉnh đích, toàn bộ các cạnh và đỉnh cấu thành đường đi ngắn nhất tự động chuyển đổi màu sắc hiển thị sang màu đỏ với độ dày lớn hơn nhằm làm rõ lộ trình trực quan hơn, dễ quan sát hơn.



Hình 4.4: Biểu diễn lộ trình ngắn nhất bằng hiệu ứng đổi màu cung liên kết sang sắc đỏ

4.3.3. Nhận xét đánh giá

Ưu điểm:

- Chương trình chạy ổn định, giải quyết đúng yêu cầu bài toán tìm đường đi ngắn nhất mà đề tài đặt ra.
- Giao diện đồ họa sinh động, dễ nhìn. Thay vì phải đọc những mảng số khô khan in ra từ màn hình Console, người dùng có thể quan sát thuật toán loang từng bước một, rất phù hợp để làm công cụ hỗ trợ ôn tập môn Toán rời rạc.
- Nhóm chúng em sử dụng mảng tĩnh (cấp phát bộ nhớ sẵn) nên chương trình rất nhẹ, dễ kiểm soát lỗi và không bị tình trạng tràn bộ nhớ (Memory Leak).

Nhược điểm:

- **Cách nhập dữ liệu còn cứng nhắc:** Hiện tại, người dùng bắt buộc phải tự tạo file text và gõ ma trận kề cho đúng định dạng thì chương trình mới chạy được. Nếu gõ sai hoặc dư dấu cách, chương trình dễ bị lỗi.
- **Chưa tối ưu hóa hoàn toàn về Cấu trúc dữ liệu liên kết:** Mặc dù đã tích hợp thành công Hàng đợi ưu tiên (Min-Heap) để chọn đỉnh với tốc độ $O(\log V)$, thuật toán vẫn còn sử dụng Ma trận kề làm hệ thống lưu trữ gốc, khiến bước nối lỏng cạnh vẫn phải quét mảng với chi phí tổng cộng $O(V^2)$.

Hướng cải tiến: Bổ sung thêm các đoạn code kiểm tra điều kiện đầu vào (bắt lỗi file rỗng, lỗi nhập chữ thay vì số) để hiện thông báo nhắc nhở người dùng thay vì thoát phần mềm đột ngột. Nhóm sẽ bổ sung thêm tùy chọn cho phép người dùng linh hoạt chuyển đổi giữa việc sử dụng Ma trận kề (Adjacency Matrix) hoặc Danh sách kề (Adjacency List) tùy thuộc vào kích thước và mật độ của đồ thị. Khi kết hợp Danh sách kề với cấu trúc Min-Heap hiện có, hệ thống sẽ đạt độ phức tạp lý tưởng $O((V + E) \log V)$, giúp phần mềm chạy mượt mà kể cả khi bản đồ có hàng vạn đỉnh.

5. Kết luận và hướng phát triển

5.1. Kết luận

Qua quá trình nỗ lực nghiên cứu và hoàn thiện đồ án PBL1, nhóm chúng em đã củng cố thành công nền tảng cấu trúc dữ liệu tĩnh và làm chủ giải thuật cốt lõi Dijkstra. Việc áp dụng thành công kỹ thuật hình học lượng giác và thư viện Raylib vào kiến trúc Máy trạng thái phân mảnh đã giúp chương trình vận hành chính xác, minh họa trực quan sinh động tiến trình loang nhãn phức tạp của hệ thống đồ thị.

5.2. Hướng phát triển

Dựa trên những ưu điểm về mặt đồ họa đã làm được, nhóm chúng em dự định sẽ tiếp tục nâng cấp đồ án theo các hướng sau nếu có thêm thời gian:

- **Thêm chức năng vẽ bằng chuột:** Thay vì phải gõ từng con số vào file văn bản, tụi em muốn nâng cấp để người dùng có thể dùng chuột click lên màn hình để tạo đỉnh, sau đó kéo thả để nối các đường đi và nhập trọng số trực tiếp.
- **Tích hợp thêm thuật toán khác:** Bổ sung thêm thuật toán Bellman-Ford hoặc A* (A-Star) vào chương trình. Khi đó, phần mềm sẽ cho phép người dùng chọn xem thuật toán nào, hoặc chia đôi màn hình để xem hai thuật toán "chạy đua" tìm đường cùng một lúc.
- **Cải thiện giao diện người dùng (UI):** Thiết kế thêm các nút bấm (Button) trên màn hình để Play, Pause hoặc Tua nhanh thuật toán, thay vì phải ghi nhớ và bấm các phím tắt trên bàn phím như hiện tại.

Tài liệu tham khảo

- [1] Phan Thanh Tao, *Giáo trình Toán rời rạc*, Trường Đại học Bách Khoa – Đại học Đà Nẵng.
- [2] Raylib Graph Graphics Development Documentation, <https://www.raylib.com/>.
- [3] Nguyễn Văn Hiệu, *Bài toán đường đi ngắn nhất trên đồ thị*, Trường Đại học Bách Khoa - Đại học Đà Nẵng.
- [4] Nguyễn Văn Hiệu, *Tập bài giảng Toán rời rạc*, Trường Đại học Bách Khoa - Đại học Đà Nẵng.

PHỤ LỤC

MÃ NGUỒN DỰ ÁN (GITHUB)

Toàn bộ mã nguồn, cách cài đặt, cách sử dụng đồ án được lưu trữ tại:

<https://github.com/raii-cod/PBL-1>

Trích đoạn các hàm xử lý cốt lõi của ứng dụng

1. Động cơ phân mảnh Máy trạng thái Dijkstra:

```
1 if (state == STATE_FIND_MIN) {
2     int min_d = INF; currentV = -1;
3     while (q.heap_size > 0) {
4         Node top = pop_min_heap(q.A, &q.heap_size);
5         if (g.T[top.id]) {
6             min_d = top.dist;
7             currentV = top.id;
8             break;
9         }
10    }
11
12    if (currentV == -1 || currentV == v_target) {
13        state = STATE_FINISHED;
14        if (g.L[v_target] != INF) {
15            int t = v_target;
16            int temp[N], count = 0;
17            while (t != -1) { onPath[t] = true; temp[count++] = t; t =
18                g.par[t]; }
19            AddLog(">>> KET THUC: Da xac dinh lo trinh.", stepLogs, &
20                logCount);
21        } else {
22            g.T[currentV] = 0; neighborIdx = 1; state = STATE_RELAX_EDGES;
23        }
24    } else if (state == STATE_RELAX_EDGES) {
25        bool found = false;
26        while (neighborIdx <= g.n) {
27            if (g.A[currentV][neighborIdx] != -1 && g.T[neighborIdx]) {
```

```
28         int newD = g.L[currentV] + g.A[currentV][neighborIdx];
29         if (newD < g.L[neighborIdx]) {
30             g.L[neighborIdx] = newD;
31             g.par[neighborIdx] = currentV;
32             Node P; P.id = neighborIdx; P.dist = newD;
33             insert_min_heap(q.A, &q.heap_size, P);
34         }
35         neighborIdx++; found = true; break;
36     }
37     neighborIdx++;
38 }
39 if (!found) state = STATE_FIND_MIN;
40 }
```